HospiTrack Deployment Guide

This comprehensive guide covers deploying HospiTrack to various platforms, from cloud platforms like Render to custom VPS deployments.

Table of Contents

Prerequisites

Before deploying HospiTrack, ensure you have:

Required

- ✅ **GitHub Account** - For repository hosting
- ✅ **Git installed locally** - For version control
- ✅ **Data files ready**:
- `data/us_er.parquet` - Hospital dataset (must be present)
- `models/triage_model.pkl` - ML triage model (optional)
- `models/triage_encoders.pkl` - ML encoders (optional)

Platform-Specific

- **Render**: Free account at render.com (https://render.com)
- **Fly.io**: Account at fly.io (https://fly.io) + Fly CLI installed
- **Railway**: Account at railway.app (https://railway.app)
- **VPS**: Cloud VM (AWS EC2, DigitalOcean, etc.) with Docker installed

---

Deployment Options Overview

Platform	Difficulty	Cost	Best For
Render	⭐ Easy	$7+/mo (Starter plan)	Quick deployment, auto-scaling
Fly.io	⭐⭐ Moderate	~$5+/mo	Global edge deployment
Railway	⭐ Easy	$5+/mo	Simple git-based deploys
Docker Compose (VPS)	⭐⭐⭐ Advanced	Varies	Full control, custom setup

---

Primary: Render Deployment

Render is the **recommended** platform for HospiTrack due to its simplicity and reliability.

Step 1: Prepare Your Repository

1. **Push your code to GitHub** (if not already done):
   ```
   bash
   cd /path/to/hospitracker
   git init
   git add .
   git commit -m "Initial commit for deployment"
   git remote add origin https://github.com/YOUR_USERNAME/hospitracker.git
   git push -u origin main
   ```

2. **Verify required files are present**:
   ```
   bash
   # Check for essential files
   ls -la Dockerfile.prod render.yaml data/us_er.parquet
   ```

Step 2: Create Render Account

1. Go to render.com (https://render.com) and sign up
2. Connect your GitHub account when prompted
3. Grant Render access to your `hospitracker` repository

Step 3: Deploy Using Blueprint (Automatic)

Option A: Using render.yaml (Recommended)

1. In Render Dashboard, click **“New +” → “Blueprint”**
2. Select your `hospitracker` repository
3. Render will automatically detect `render.yaml`

4. Review the configuration:
   - Service name: `hospitracker`
   - Docker image: `Dockerfile.prod`
   - Instance type: Starter ($7/mo)
5. Click **“Apply”** to create the service
6. Render will:
   - Build the Docker image
   - Deploy to their infrastructure
   - Assign a public URL (e.g., `https://hospitracker.onrender.com`)

Option B: Manual Setup

If you prefer manual configuration or want to customize:

1. Click **“New +”** → **“Web Service”**
2. Connect your GitHub repository
3. Configure the service:
   - **Name**: `hospitracker`
   - **Region**: Choose closest to your users (e.g., Oregon, Frankfurt)
   - **Branch**: `main`
   - **Runtime**: Docker
   - **Dockerfile Path**: `./Dockerfile.prod`
   - **Docker Build Context Directory**: `.`
4. Set instance type: **Starter** (1 GB RAM, $7/mo minimum recommended)
5. Click **“Create Web Service”**

Step 4: Configure Environment Variables

After deployment starts, add environment variables:

1. Go to your service → **“Environment”** tab
2. Add the following variables:

Key	Value	Description
`PORT`	`8000`	Application port
`PYTHONUNBUFFERED`	`1`	Enable real-time logging
`HOSPITRACK_DATA_PATH`	`/app/data`	Data directory path
`GEOCODING_CACHE_SIZE`	`1000`	Geocoding cache size
`ML_DEMO_ENABLED`	`true`	Enable ML triage demo
`LOG_LEVEL`	`INFO`	Logging verbosity

1. Click **“Save Changes”** (will trigger auto-redeploy)

Step 5: Verify Deployment

1. Wait for build to complete (5-10 minutes for first build)
2. Check **“Logs”** tab for any errors

3. Once deployed, visit your public URL
4. Test the following:
   - Landing page loads: `https://your-app.onrender.com/`
   - API docs work: `https://your-app.onrender.com/docs`
   - Health check: `https://your-app.onrender.com/healthz`
   - Search functionality: Try searching for hospitals

Step 6: Set Up Auto-Deployment

Render automatically deploys on git push if configured:

1. Go to **“Settings” → “Build & Deploy”**
2. Ensure **“Auto-Deploy”** is enabled for `main` branch
3. Now every `git push` will trigger a new deployment

Optional: Configure Custom Domain

1. In service settings, go to **“Custom Domains”**
2. Click **“Add Custom Domain”**
3. Enter your domain (e.g., `hospitracker.com`)
4. Add the provided DNS records to your domain registrar
5. Wait for DNS propagation (up to 48 hours)
6. Render automatically provisions SSL/TLS certificates

Optional: Enable Persistent Disk (If Needed)

If you need to store data outside the Docker image:

1. Go to **“Disks”** tab
2. Click **“Add Disk”**
3. Configure:
   - **Name**: `data`
   - **Mount Path**: `/app/data`
   - **Size**: 1 GB (adjust as needed)
4. Modify your code to write data to `/app/data`

Alternative: Fly.io Deployment

Fly.io offers global edge deployment with data center distribution.

Prerequisites

1. Install Fly CLI:
   `bash`
   `curl -L https://fly.io/install.sh | sh`
2. Login to Fly:
   `bash`
   `fly auth login`

Deployment Steps

1. **Create fly.toml configuration**:

Create `fly.toml` in your project root:
```toml
app = “hospitracker”

[build]
dockerfile = “Dockerfile.prod”

[env]
PORT = “8000”
PYTHONUNBUFFERED = “1”
HOSPITRACK_DATA_PATH = “/app/data”
GEOCODING_CACHE_SIZE = “1000”
ML_DEMO_ENABLED = “true”
LOG_LEVEL = “INFO”

[[services]]
internal_port = 8000
protocol = “tcp”
```

```
  [services.concurrency]
    hard_limit = 200
    soft_limit = 100

  [[services.ports]]
    handlers = ["http"]
    port = 80
    force_https = true

  [[services.ports]]
    handlers = ["tls", "http"]
    port = 443

  [[services.tcp_checks]]
    grace_period = "10s"
    interval = "30s"
    restart_limit = 0
    timeout = "5s"

  [[services.http_checks]]
    interval = "30s"
    grace_period = "10s"
    method = "get"
    path = "/healthz"
    protocol = "http"
    restart_limit = 0
    timeout = "5s"
```

```

1. **Launch the app**:
   ```bash
   fly launch –no-deploy # Configure without deploying
   # Follow prompts to select region, instance size, etc.

fly deploy # Deploy the application
```

1. **Open the app**:
   `bash`
   `fly open`

2. **View logs**:
   `bash`
   `fly logs`

Fly.io Scaling

```
# Scale to 2 instances
fly scale count 2

# Scale memory
fly scale memory 1024  # 1 GB RAM
```

Alternative: Railway Deployment

Railway offers simple git-based deployments.

Deployment Steps

1. **Go to railway.app (https://railway.app)** and sign up
2. Click **“New Project” → “Deploy from GitHub repo”**
3. Select your `hospitracker` repository
4. Railway will auto-detect Docker configuration
5. **Configure environment variables**:
   - Go to **“Variables”** tab
   - Add the same variables as Render deployment
6. **Configure deployment settings**:

Create `railway.json` (optional):
`json`
```
  {
    "$schema": "https://railway.app/railway.schema.json",
    "build": {
      "builder": "DOCKERFILE",
      "dockerfilePath": "Dockerfile.prod"
    },
    "deploy": {
      "startCommand": "",
      "healthcheckPath": "/healthz",
      "healthcheckTimeout": 100,
      "restartPolicyType": "ON_FAILURE",
      "restartPolicyMaxRetries": 10
```
```

```
    }
  }
```

1. Railway will automatically deploy and provide a public URL

Alternative: Docker Compose on VPS

For custom deployments on your own server (AWS EC2, DigitalOcean, Linode, etc.).

Prerequisites

- VPS with Ubuntu 20.04+ or similar
- Docker and Docker Compose installed
- Domain name pointed to VPS IP (optional but recommended)

Step 1: Install Docker

```
# Update system
sudo apt-get update && sudo apt-get upgrade -y

# Install Docker
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh

# Install Docker Compose
sudo curl -L "https://github.com/docker/compose/releases/latest/download/docker-com-
pose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
sudo chmod +x /usr/local/bin/docker-compose

# Add your user to docker group
sudo usermod -aG docker $USER
newgrp docker
```

Step 2: Clone Repository

```
cd /opt
sudo git clone https://github.com/YOUR_USERNAME/hospitracker.git
cd hospitracker
```

Step 3: Use Production Docker Compose

Create `docker-compose.prod.yml`:

```yaml
version: "3.8"

services:
  hospitrack:
    build:
      context: .
      dockerfile: Dockerfile.prod
    image: hospitracker:production
    container_name: hospitracker
    restart: always
    ports:
      - "8000:8000"
    environment:
      - PORT=8000
      - PYTHONUNBUFFERED=1
      - HOSPITRACK_DATA_PATH=/app/data
      - GEOCODING_CACHE_SIZE=1000
      - ML_DEMO_ENABLED=true
      - LOG_LEVEL=INFO
      - GUNICORN_WORKERS=4
    volumes:
      - ./data:/app/data:ro
      - ./models:/app/models:ro
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/healthz"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 40s
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"
    networks:
      - hospitrack-network

networks:
  hospitrack-network:
    driver: bridge
```

Step 4: Deploy

```bash
# Build and start
docker-compose -f docker-compose.prod.yml up -d --build

# Check logs
docker-compose -f docker-compose.prod.yml logs -f

# Verify health
curl http://localhost:8000/healthz
```

Step 5: Set Up Nginx Reverse Proxy

1. **Install Nginx**:
   `bash`
   `sudo apt-get install nginx -y`

2. **Create Nginx configuration**:
   `bash`
   `sudo nano /etc/nginx/sites-available/hospitracker`

Add:
```nginx
server {
listen 80;
server_name your-domain.com www.your-domain.com;

    client_max_body_size 10M;

    location / {
        proxy_pass http://localhost:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # WebSocket support (if needed)
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";

        # Timeouts
        proxy_connect_timeout 60s;
        proxy_send_timeout 60s;
        proxy_read_timeout 60s;
    }

    # Health check endpoint
    location /healthz {
        proxy_pass http://localhost:8000/healthz;
        access_log off;
    }

}
```

1. **Enable the site**:
   `bash`
   `sudo ln -s /etc/nginx/sites-available/hospitracker /etc/nginx/sites-enabled/`
   `sudo nginx -t  # Test configuration`
   `sudo systemctl restart nginx`

Step 6: Set Up SSL with Let’s Encrypt

```
# Install Certbot
sudo apt-get install certbot python3-certbot-nginx -y

# Obtain SSL certificate
sudo certbot --nginx -d your-domain.com -d www.your-domain.com

# Auto-renewal is set up automatically
# Test renewal:
sudo certbot renew --dry-run
```

Step 7: Set Up Firewall

```
# Allow SSH, HTTP, HTTPS
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status
```

Custom VPS/Cloud Deployment

AWS EC2 Deployment

1. **Launch EC2 instance**:
   - AMI: Ubuntu 20.04 LTS
   - Instance type: t3.small (minimum) or t3.medium (recommended)
   - Storage: 20 GB gp3
   - Security group: Allow ports 22, 80, 443

2. **Connect to instance**:
   `bash`
   `ssh -i your-key.pem ubuntu@your-ec2-ip`

3. Follow Docker Compose on VPS steps above

DigitalOcean Deployment

1. **Create Droplet**:
   - Image: Docker on Ubuntu 20.04
   - Size: Basic plan, 2 GB RAM minimum
   - Add SSH key

2. **Connect and deploy**:
   `bash`
   `ssh root@your-droplet-ip`

3. Follow Docker Compose on VPS steps

Google Cloud Platform (GCP)

1. **Create Compute Engine VM**:
   - Machine type: e2-small (minimum)
   - Boot disk: Ubuntu 20.04 LTS, 20 GB
   - Firewall: Allow HTTP/HTTPS traffic

2. **Connect via SSH** (from GCP Console or gcloud CLI)

3. Follow Docker Compose on VPS steps

Post-Deployment Verification

After deploying to any platform, verify everything works:

1. Health Check

```
curl https://your-app-url.com/healthz
# Expected: {"status": "healthy"}
```

2. API Documentation

Visit: `https://your-app-url.com/docs`
- Should display FastAPI Swagger UI
- Try the interactive API tester

3. Test Core Functionality

Test Search Endpoint:

```
curl -X POST https://your-app-url.com/api/search \
  -H "Content-Type: application/json" \
  -d '{
    "complaint": "chest pain",
    "priority": "fastest_care",
    "user_location": "San Francisco, CA",
    "radius_km": 25
  }'
```

Test Explore Endpoint:

```
curl "https://your-app-url.com/api/explore?state=CA&limit=10"
```

Test Triage Endpoint:

```
curl -X POST https://your-app-url.com/api/triage \
  -H "Content-Type: application/json" \
  -d '{
    "complaint": "chest pain",
    "severity": 4,
    "age_band": "adult",
    "heart_rate": 110,
    "use_ml": false
  }'
```

4. Frontend Testing

1. Visit the landing page: `https://your-app-url.com/`
2. Test navigation to:
   - Home search: `/static/home.html`
   - Explore: `/static/explore.html`
   - Company demo: `/static/demo.html`
3. Test hospital search with real locations
4. Verify map rendering and interactivity

5. Performance Check

```
# Load testing (optional)
ab -n 100 -c 10 https://your-app-url.com/healthz
```

Troubleshooting

Common Issues

1. Build Fails: “No module named ‘fastapi’“

Solution: Ensure `requirements_fastapi.txt` is correctly copied in Dockerfile.

2. Health Check Fails

Solution:
- Verify `/healthz` endpoint exists in `main.py`
- Check if app is listening on correct port
- Review logs: `docker logs <container-id>`

3. Data File Not Found

Solution:
- Verify `data/us_er.parquet` exists in repository
- Check `.dockerignore` isn’t excluding data files
- Ensure data directory is copied in Dockerfile

4. High Memory Usage

Solution:
- Reduce Gunicorn workers: Set `GUNICORN_WORKERS=2`
- Upgrade to instance with more RAM
- Optimize data loading in `data_loader.py`

5. Geocoding Rate Limit Errors

Solution:
- Increase `GEOCODING_CACHE_SIZE` to 5000
- Implement request throttling on frontend
- Consider using commercial geocoding API

6. Slow Initial Load

Solution:
- Parquet file is already optimized
- Ensure data is bundled in Docker image, not mounted
- Increase health check `start_period` to 60s

Platform-Specific Issues

Render

- **Build timeout**: Increase to 20 minutes in Settings
- **Out of memory**: Upgrade to Standard plan (2 GB RAM)

Fly.io

- **Region issues**: Use `fly regions add <region>` to add more
- **Certificate errors**: Run `fly certs check`

Railway

- **Deploy fails**: Check build logs in Railway dashboard
- **Domain issues**: Verify DNS settings in Railway

Debugging Commands

```
# Check container logs
docker logs hospitracker -f --tail 100

# Check resource usage
docker stats hospitracker

# Enter container shell
docker exec -it hospitracker /bin/bash

# Test health endpoint inside container
docker exec hospitracker curl http://localhost:8000/healthz

# Verify data files exist
docker exec hospitracker ls -la /app/data
```

Getting Help

- **GitHub Issues**: Open an issue with logs and error messages
- **Render Support**: support@render.com
- **Community**: Stack Overflow tag `hospitrack` or `fastapi`

Next Steps

After successful deployment:

1. ✅ Set up monitoring (see PRODUCTION_CHECKLIST.md (./PRODUCTION_CHECKLIST.md))
2. ✅ Configure backup strategy for data
3. ✅ Set up error tracking (Sentry, etc.)
4. ✅ Enable analytics (optional)
5. ✅ Create staging environment for testing
6. ✅ Set up CI/CD pipeline (see GitHub Actions workflow)

Additional Resources

- FastAPI Deployment Documentation (https://fastapi.tiangolo.com/deployment/)
- Docker Best Practices (https://docs.docker.com/develop/dev-best-practices/)
- Nginx Configuration Guide (https://nginx.org/en/docs/)
- Let’s Encrypt Documentation (https://letsencrypt.org/docs/)

Last Updated: December 2024
Maintained by: HospiTrack Team